

1. 采用超链接的解析，有没有可能访问非法目录

有可能，后续是通过解析成绝对路径，然后对绝对路径和给定的目录做前缀匹配来判断是否是合法路径

2. 模式

头文件：定义数据结构、声明函数接口

源文件：实现头文件声明的函数，包含具体业务逻辑

3. channel 的作用

封装文件描述符 `fd`，并管理对应的读写事件

```
// 定义函数指针
typedef int(*handleFunc)(void* arg);
struct Channel {
    int fd; // 文件描述符
    int events; // 监听的事件标志位（枚举类型，读/写/超时）
    handleFunc readCallback; // 读事件回调，在fd可读时触发，函数指针
    handleFunc writeCallback; // 写事件回调，在fd可写时触发，一般都是可写的
    handleFunc destroyCallback;
    void* arg; // 回调参数
};
// 修改fd的写事件(检测 or 不检测)
void writeEventEnable(struct Channel* channel, bool flag);
// 判断是否需要检测文件描述符的写事件
bool isWriteEventEnable(struct Channel* channel);
```

与 `Reactor` 协作，动态管理时间的监听状态

`EventLoop` 通过 `channel` 注册 `fd` 到多路复用器 `epoll`、`poll`、`select`，当事件就绪的时候，`EventLoop` 调用对应的 `Channel` 回调函数

4. buffer 的作用

动态内存管理（初始化、销毁、扩容机制），追踪读写位置，数据读写，数据发送

```
struct Buffer
{
    // 指向内存的指针
    char* data;
    int capacity;
    int readPos;
    int writePos; // 已读可写 --- ▲读指针▲ --- 未读已写 --- ▲写指针▲ --- 未读未写
    --- ▲容量指针▲
};
```

扩容机制

1. 直接剩余空间足够：不扩容
2. 合并已读空间后足够（读指针+`bufferwritable`：容量指针-写指针）：移动数据到头部（`memcpy`），腾出的位置做复用空间

3. 必须扩容: `realloc` 扩容

写数据的方式

追加二进制数据 (先做预扩容判断): 对 `buf` 的写指针, 执行 `memcpy`

追加字符串 (基于二进制数据封装)

使用 `readv` 双缓冲区从 `socket-fd` 中读取数据到缓冲区 (支持分散写入)

```
struct iovec vec[2];
// 初始化数组元素
int writeable = bufferwriteableSize(buffer);
vec[0].iov_base = buffer->data + buffer->writePos;
vec[0].iov_len = writeable;
char* tmpbuf = (char*)malloc(40960);
vec[1].iov_base = tmpbuf; // 如果数据量很大, 需要使用临时缓冲区40k
vec[1].iov_len = 40960;
int result = readv(fd, vec, 2);
/* 根据写入到vec的结果判断是否需要使用到第二个缓冲区, 做buf重分配
else
{
    buffer->writePos = buffer->capacity;
    bufferAppendData(buffer, tmpbuf, result - writeable);
} */
```

使用 `memmem` 在二进制数据块中查找 `\r\n`, 用于 `http` 解析

```
char* ptr = memmem(buffer->data + buffer->readPos,
bufferReadableSize(buffer), "\r\n", 2);
```

发送数据中, 采用 `send`, 同时 `MSG_NOSIGNAL` 避免在写入已关闭 `socket` 时产生终止信号

5. `channelMap` 的作用

通过动态数组 (`struct channel** list`) 实现 `fd` 到 `channel*` 的直接映射 (哈希表, 键: `fd`, 值: `channel` 对象指针)

```
struct ChannelMap
{
    int size; // 记录指针指向的数组的元素总个数
    // struct Channel* list[];
    struct Channel** list;
};
```

动态扩容: 每次按照当前容量的2倍扩容, 直至满足 `newSize`, 扩容成功初始化新空间 `NULL`

```
if (map->size < newSize)
{
    int curSize = map->size;
    // 容量每次扩大原来的一倍
    while (curSize < newSize)
        curSize *= 2;
    // 扩容 realloc
    struct Channel** temp = realloc(map->list, curSize * unitSize);
```

```

    if (temp == NULL)
    {
        return false;
    }
    map->list = temp;
    memset(&map->list[map->size], 0, (curSize - map->size) * unitSize);
    // 扩容区初始化
    map->size = curSize;
}

```

这样的哈希结构，使得在 `EventLoop` 中可以使用 `直接索引` 来快速查找 `channel`，既省去了哈希计算，也避免了哈希冲突

```

// EventLoop.c 中操作示例
if (fd < channelMap->size) {
    struct Channel* channel = channelMap->list[fd];
}

```

6. Dispatcher 的作用

负责事件的多路分发与处理，抽象了不同的多路复用机制（`epoll`、`poll`、`select`），以函数指针接口的方式，方便上层实现高效的事件监听和调度

将 `epoll`、`poll`、`select` 的差异隐藏在底层，对外提供一致的接口

```

struct EventLoop;
struct Dispatcher
{
    // init -- 初始化epoll, poll 或者 select 需要的数据块
    void* (*init)();
    // 添加: 将fd及关注的事件注册到多路复用器
    int (*add)(struct Channel* channel, struct EventLoop* evLoop);
    // 删除
    int (*remove)(struct Channel* channel, struct EventLoop* evLoop);
    // 修改: 修改fd的监听事件（启动、禁用写事件）
    int (*modify)(struct Channel* channel, struct EventLoop* evLoop);
    // 事件监测
    int (*dispatch)(struct EventLoop* evLoop, int timeout); // 单位: s
    // 清除数据(关闭fd或者释放内存)
    int (*clear)(struct EventLoop* evLoop);
};
// 定义结构体之后，只需实现函数细节
struct Dispatcher epollDispatcher = {
    epollInit, epollAdd, epollRemove, epollModify, epollDispatch,
    epollClear
};

```

7. ThreadPool 的作用

线程池，由主线程调用，基于主从 `Reactor` 模型实现，用于高效管理多个子线程的事件循环，提升网络服务器的并发处理能力

```

struct ThreadPool
{
    // 主线程的反应堆模型
    struct EventLoop* mainLoop;
    bool isStart;
    int threadNum;
    struct WorkerThread* workerThreads; // 子线程数组
    int index;
};

```

初始化线程池、启动线程池

主线程根据传入的 `EventLoop` 和 `cnt` 来初始化线程池，分配 `cnt` 个工作线程，主线程传入 `EventLoop` 用于监听新连接

根据线程池状态来启动线程池，对工作线程数组（子线程）进行初始化和启动

获取线程的 `EventLoop`，以轮询的策略，将任务分配给子线程

```

struct EventLoop* evLoop = pool->mainLoop; // 先初始化成主线程的evLoop（在没有子线程的时候，主线程执行任务）
if (pool->threadNum > 0)
{
    evLoop = pool->workerThreads[pool->index].evLoop; // 有子线程的时候使用子线程的evloop
    pool->index = ++pool->index % pool->threadNum;
}

```

8. EventLoop 的作用

反应堆实例（网络模型的事件循环）：每个 `EventLoop` 运行在一个独立的线程中，负责监听并处理一组 `fd` 事件

```

struct EventLoop
{
    bool isQuit;
    struct Dispatcher* dispatcher; // 多路复用的具体方式（事件分发器，此处用select）
    void* dispatcherData; // 分发器私有数据
    // 任务队列指针
    struct ChannelElement* head;
    struct ChannelElement* tail;
    // map
    struct ChannelMap* channelMap;
    // 线程id, name, mutex
    pthread_t threadID;
    char threadName[32];
    pthread_mutex_t mutex;
    int socketPair[2]; // 存储本地通信的fd 通过socketpair 初始化，进程间通信管道
};

```

事件队列中的节点：

```
// 定义任务队列的节点
struct ChannelElement
{
    int type;    // 如何处理该节点中的channel
    struct Channel* channel;
    struct ChannelElement* next;
};
// 处理该节点中的channel的方式
enum ElemType{ADD, DELETE, MODIFY};
```

创建 EventLoop 实例

选择具体的多路复用器，默认 `SelectDispatcher`

创建 `fd` 到 `Channel` 的映射

创建一对用于线程间通信的管道 `fd`，将管道读端注册到 `EventLoop`，用于接收其他线程的任务

```
evLoop->dispatcher = &SelectDispatcher;
evLoop->dispatcherData = evLoop->dispatcher->init();
// map
evLoop->channelMap = channelMapInit(128);
int ret = socketpair(AF_UNIX, SOCK_STREAM, 0, evLoop->socketPair);
// 指定规则: evLoop->socketPair[0] 发送数据, evLoop->socketPair[1] 接收数据
struct Channel* channel = channelInit(evLoop->socketPair[1],
ReadEvent, readLocalMessage, NULL, NULL, evLoop);
// channel 添加到任务队列
eventLoopAddTask(evLoop, channel, ADD);
```

启动事件循环

`dispatch` 阻塞等待 `fd` 事件就绪或者超时，然后去处理线程间传递过来的任务

```
while (!evLoop->isQuit)
{
    dispatcher->dispatch(evLoop, 2);    // 超时时长 2s
    eventLoopProcessTask(evLoop);
}
```

事件激活与处理

当多路复用器报告 `fd` 事件就绪时，`Dispatcher` 调用，根据就绪事件类型（读写）调用 `channel` 中注册的回调函数

```
struct Channel* channel = evLoop->channelMap->list[fd];
assert(channel->fd == fd);
if (event & ReadEvent && channel->readCallback)
    channel->readCallback(channel->arg);
if (event & WriteEvent && channel->writeCallback)
    channel->writeCallback(channel->arg);
```

任务队列：使用链表存储需要在 `EventLoop` 线程中执行的任务（添加、删除、修改 `channel`）

添加任务，最终都落实到某个线程调用 `eventLoopProcessTask` 做实际处理

```
// 加锁，创建一个任务节点，添加到任务队列中，根据线程id来处理
int eventLoopAddTask(struct EventLoop* evLoop, struct Channel*
channel, int type) {
    pthread_mutex_lock(&evLoop->mutex); // 加锁保护任务队列
    // 添加任务节点到链表
    pthread_mutex_unlock(&evLoop->mutex);
    if (evLoop->threadID == pthread_self()) {
        eventLoopProcessTask(evLoop); // 若在当前线程，直接处理
    } else {
        taskwakeup(evLoop);           // 若在其他线程（如主线程），唤醒
EventLoop 线程处理
    }
    return 0;
}
```

```
void taskwakeup(struct EventLoop* evLoop) {
    write(evLoop->socketPair[0], "...", 1); // 向管道写入数据，唤醒阻塞在
dispatch 的 EventLoop 线程
}
int readLocalMessage(void* arg) {
    // 读取管道数据，处理唤醒
    read(evLoop->socketPair[1], buf, sizeof(buf));
    eventLoopProcessTask(evLoop); // 处理任务队列
    return 0;
}
```

实际处理任务队列中的任务

```
int eventLoopProcessTask(struct EventLoop* evLoop) {
    pthread_mutex_lock(&evLoop->mutex);
    // 遍历任务队列，根据任务类型（ADD/DELETE/MODIFY）调用 Dispatcher 对应函
数
    pthread_mutex_unlock(&evLoop->mutex);
    return 0;
}
```

cursor 总结

`EventLoop` 的核心作用：

1. 事件驱动：是 `Reactor` 模型的核心驱动力，通过事件循环处理 `I/O` 事件
2. 单线程处理：每个 `EventLoop` 在一个线程中运行，避免多线程竞争，简化编程模型
3. 职责集中：继承了多路复用、事件分发、任务调度、线程间通信
4. 模块协作：与事件处理器、多路复用抽象、`fd` 管理紧密协作

总结：

`EventLoop` 是反应堆模型的事件处理引擎，能够

- 持续监听并处理注册的 fd 事件
- 同任务队列和线程间通信机制处理跨线程操作
- 在事件就绪时，调用 channel 中注册的回调函数执行业务逻辑

9. SelectDispatcher 的作用

`select`：用于实现 I/O 多路复用，允许一个进程监视多个 FD，以查看他们之间是否有一个或多个已经准备好进行某种 I/O 操作（读、写、异常）。主要关注的对象是 `fd_set` 的文件描述符集合，这是一个位图，每一位对应一个文件描述符，并且提供了一系列宏操作来管理

`FD_ZERO`、`FD_SET`、`FD_CLR`、`FD_ISSET`

```
int select(int nfds, fd_set *readfds, fd_set *writefds, fd_set *exceptfds,
struct timeval *timeout);
```

其中的关于 `fd_set` 的参数都是传入传出参数，内核在检测的时候，会检测对应的读写缓冲区，传出已就绪的 fd，作为判断依据，因此需要传入的是一个副本

最后一个参数是一个带微秒、秒的超时参数，不指定代表阻塞等待就绪，0则立即返回

`fd_set` 待检测的文件描述符通常被限制为1024，这意味着 `select` 不能有效处理超过这个数量的 fd，但可以修改上限

其每次都需要根据传入的第一个参数来决定遍历的范围，但通常是遍历整个 `fd_set`，这使得在 fd 很多，而活跃的很少时，也需要全部遍历，开销很大

`SelectDispatcher`：基于 `select` 实现了 `Dispatcher` 接口，作为底层的 I/O 多路复用机制其所需的数据无非是读写事件的文件描述符集，每个 `EventLoop` 的 `dispatcherData` 都会指向这样一个结构实例

```
struct SelectData
{
    fd_set readSet; // 监视可读事件的文件描述符集合
    fd_set writeSet; // 监视可写事件的文件描述符集合
};
```

提供了辅助函数来操作，也就是根据和 `channel` 封装的 fd 待检测的事件做与运算，修改传入的 `SelectData` 对应的事件

```
static void setFdSet(struct Channel* channel, struct SelectData* data)
{
    // 检查channel关注的事件，如果是读，就将fd加入readSet，如果是写，就加入writeSet
    if (channel->events & ReadEvent)
        FD_SET(channel->fd, &data->readSet);
    if (channel->events & WriteEvent)
        FD_SET(channel->fd, &data->writeSet);
}

static void clearFdSet(struct Channel* channel, struct SelectData* data)
{
    if (channel->events & ReadEvent)
        FD_CLR(channel->fd, &data->readSet);
    if (channel->events & WriteEvent)
        FD_CLR(channel->fd, &data->writeSet);
}
```

初始化函数就是构造一个 `data` 实例并返回

提供了添加、删除、修改 `channel` 中 `fd` 到相应 `fd_set` 中的函数，删除的时候需要调用 `channel` 中的销毁回调函数，修改的时候，在确保了当前传入的 `channel` 对应的 `event` 是最新状态，然后无条件修改 `channel` 中的 `fd`，并调用 `setFdSet` 去更新 `channel` 的 `event`

核心处理：事件分发

```
static int selectDispatch(struct EventLoop* evLoop, int timeout)
{
    struct SelectData* data = (struct SelectData*)evLoop->dispatcherData;
    struct timeval val; val.tv_sec = timeout; val.tv_usec = 0;
    // 因为select会修改传入的fd_set，所以需要拷贝一份
    fd_set rdtmp = data->readSet; fd_set wrtmp = data->writeSet;
    int count = select(Max, &rdtmp, &wrtmp, NULL, &val); // 调用select
    // 遍历所有可能的fd，检查哪些是就绪的
    for (int i = 0; i < Max; ++i)
    {
        if (FD_ISSET(i, &rdtmp)) // 检查是否在返回的读集合中
            eventActivate(evLoop, i, ReadEvent); // 激活读事件
        if (FD_ISSET(i, &wrtmp)) // 检查是否在返回的写集合中
            eventActivate(evLoop, i, WriteEvent); // 激活写事件
    }
    return 0;
}
```

获取 `EventLoop` 中的 `select` 所需的 `data`，构造一个超时结构体，拷贝原先待检测的 `fd`，然后调用 `select` 去阻塞等待内核检测事件，根据传出的读写文件描述符集合，去遍历文件描述符集合与传出的就绪 `fd` 比较，激活对应的 `fd` 对应的读写事件

10. `HttpResponse` 的作用

负责构建和组织 `http` 响应，当服务器处理完一个请求后，需要向客户端发送一个 `http` 的响应

```
// 响应数据的格式：http版本 http响应码 http响应状态描述 响应头 空行 响应体
enum HttpStatusCode
{
    // 定义状态码枚举
    Unknown, OK = 200, MovedPermanently = 301, MovedTemporarily = 302,
    BadRequest = 400, NotFound = 404
};

struct ResponseHeader
{
    // 定义响应头的结构体
    char key[32];
    char value[128];
};

// 定义一个函数指针，用来组织要回复给客户端的数据块
typedef void (*responseBody)(const char* fileName, struct Buffer* sendBuf,
int socket);

struct HttpResponse
{
    // 状态行：版本号 状态码 状态描述
    enum HttpStatusCode statusCode;
    char statusMsg[128];
    char fileName[128];
    // 响应头 - 键值对
    struct ResponseHeader* headers; // content-type, content-length,
    int headerNum;
};
```



```
responseBody sendDataFunc; // 上层设置好这个指针，就可以发送不同类型的响应体（文件、目录列表等）进行处理
};
```

提供的函数是：初始化一个 `HttpResponse` 实例，释放初始化资源，添加一个响应头键值对，组装完整的 `http` 响应并发送响应体

组装 `http` 响应

准备状态行(`sprintf` + `bufferAppendString`)

准备响应头和空行(`bufferAppendString`)

可选立即发送已组装好的头部信息，拆分了数据

发送响应体（调用对应的函数指针 `response->sendDataFunc()`，追加响应体数据，发送文件、目录）

11. HttpRequest 的作用

实现关于HTTP请求的解析和处理模块

`HttpRequest` 结构体：

HTTP请求行： 请求方法 + URL + HTTP版本

请求头结构体数组（及已解析请求头数量）： 键值对

解析状态（枚举）： 请求行、请求头、请求体（被弱化）、完成

```
struct RequestHeader
{
    // 请求头键值对
    char* key;
    char* value;
};

enum HttpRequestState
{
    // 当前的解析状态
    ParseReqLine,
    ParseReqHeaders,
    ParseReqBody,
    ParseReqDone
};

struct HttpRequest
{
    char* method;
    char* url;
    char* version;
    struct RequestHeader* reqHeaders;
    int reqHeadersNum;
    enum HttpRequestState curState;
};
```

提供的函数

初始化：创建并初始化 `HttpRequest` 结构体对象，并分配固定大小 12 的请求头数组

销毁

轻量级重置：不清空已分配的请求行字符串内存，快速恢复对象

深度重置：释放堆区分配的内容（包括请求头），并调用轻量级重置

深度销毁：调用深度重置，然后释放 `HttpRequest`，彻底销毁对象

获取当前解析状态：返回当前解析状态的枚举变量

添加一个请求头键值对：添加一个请求头键值对到对象中

根据请求头的 `key` 获取 `value`：遍历请求头数组，不区分 `key` 的大小写 `strncasecmp`

解析 HTTP 请求行：从 `readBuf` 中，根据 `memmem` 去查找 `\r\n`，截断前面的请求行，然后调用辅助函数用空格分隔，提取出请求行三要素（方法、`URL`、版本），（完事之后，需要更新 `readBuf` 状态、当前解析状态）

解析单个 HTTP 请求头：

在下面的核心函数中，每次只处理一个请求头的解析，同样是截断下一个换行符，然后利用请求头的键值对特性，利用 `memmem` 来查找，然后构造出 `key` 和 `value`，调用添加函数加入对象的请求头数组中（更新状态）

另一方面，如果当前没有找到键值对，说明解析到了空行，利用 `get` 请求的处理，跳过（更新当前解析状态）

解析 `Request`（状态机、核心驱动函数）

```
bool parseHttpRequest(struct HttpRequest* request, struct Buffer*
readBuf, struct HttpResponse* response, struct Buffer* sendBuf, int
socket)
```

参数很多，主要是三部分（请求对象+读缓冲区，响应对象+写缓冲区，`socket`）

利用当前解析状态标志，循环解析，`switch-case` 来判定当前解析走向，根据解析状态选择处理请求，组织响应

```
while (request->curState != ParseReqDone)
{
    switch (request->curState)
    {
        case ParseReqLine:
            flag = parseHttpRequestLine(request, readBuf); break;
        case ParseReqHeaders:
            flag = parseHttpRequestHeader(request, readBuf); break;
        case ParseReqBody:
            // 简化了对请求体的处理，只处理get请求
            break;
        default:
            break;
    }
    if (!flag)
        return flag;
    // 判断是否解析完毕了，如果完毕了，需要准备回复的数据
    if (request->curState == ParseReqDone)
    {
        // 1. 根据解析出的原始数据，对客户端的请求做出处理
        processHttpRequest(request, response);
        // 2. 组织响应数据并发送给客户端
        httpResponsePrepareMsg(response, sendBuf, socket);
    }
}
```

```
}
```

请求处理函数

主要流程：

- 判定请求方法是否是 `get`（根据功能需求，只处理基于 `get` 的 `http` 请求）
- 将 `url` 就地原地解析（避免因为在请求中文文件目录，在传递时的路径错误）
- 根据客户端请求的 `URL` 解析成本地文件路径（`/` 和 `/xxx.c`）
- 根据请求的本地路径的文件属性，组织响应数据
 - 文件不存在，返回默认路径下的 `404.html`，组织状态码，状态信息，响应头，设置数据发送函数指针（发送文件）
 - 文件存在，判定文件类型，组织响应数据
 - 目录：利用拓展名转换类型函数，组织属于目录的 `html`，设置数据发送函数指针（发送目录）
 - 文件：组织 `content-type` 和 `content-length`，设置数据发送函数指针（发送文件）

工具函数

子字符串提取函数：从字符串中根据指定的分隔符（空格、换行符），提取子字符串（借助 `memmem` 函数），存入动态内存，返回分隔之后的下一个字符（便于链式处理）

`URL` 中文解码函数

拓展名-`MIME` 类型转换函数

目录发送函数：

利用 `scanDir` 获取给定目录下所有条目，并进行排序，存入一个文件列表明名指针数组，通过遍历，根据不同条目文件属性（目录、文件）组织不同的 `html` 行，组织一个静态网页，最后将存放静态网页标签源码的缓冲区 `buf` 数据追加到传入的发送缓冲区，等待一并发送

文件发送函数：

打开指定的文件，循环地读取文件中内容，追加到发送缓冲区，等待一并发送

12. `TcpConnection` 的作用

服务器处理单个 `Tcp` 连接

```
struct TcpConnection
{
    struct EventLoop* evLoop;
    struct Channel* channel;
    struct Buffer* readBuf;           // 用于http请求
    struct Buffer* writeBuf;         // 用于http响应
    char name[32];
    struct HttpRequest* request;
    struct HttpResponse* response;
};
```

核心：在 `tcpserver` 和客户端建立连接之后，根据传入的通信文件描述符 `fd`，在反应堆中注册一个读事件，开始监听

```
conn->channel = channelInit(fd, ReadEvent, processRead, processwrite,
tcpConnectionDestroy, conn);
eventLoopAddTask(evloop, conn->channel, ADD);
```

读、写、销毁的回调函数指针，正是 `TcpConnection` 提供的

读回调

当读事件触发时，就从对应的 `channel->fd` 中读取数据，存入读缓冲区（默认10k），然后调用 `HttpRequest` 的解析函数，解析，随之会处理请求，组织响应数据

写回调

当写事件触发时，将写缓冲区中的数据，通过 `channel->fd` 发送走，并修改检测集合，删除 `channel` 节点

销毁回调

释放对象中的所有堆内存

13. `Tcpserver` 的作用

`Tcp` 服务器的主体框架，负责监听连接、管理线程池、分发客户端连接

```
struct Listener
{
    // 封装了监听socket的基本信息，fd和port
    int lfd;
    unsigned short port;
};

struct TcpServer
{
    int threadNum;
    struct EventLoop* mainLoop;    // 主反应堆，主要负责监听新的客户端连接
    struct ThreadPool* threadPool;
    struct Listener* listener;
};
```

服务器初始化

创建 `TcpServer` 对象，初始化监听，初始化反应堆，然后根据主反应堆去初始化线程池

初始化监听

1. 创建监听的套接字：`int lfd = socket(AF_INET, SOCK_STREAM, 0);`
2. 设置端口复用：`int ret = setsockopt(lfd, SOL_SOCKET, SO_REUSEADDR, &opt, sizeof opt);`
3. 绑定

```
struct sockaddr_in addr;
addr.sin_family = AF_INET;
addr.sin_port = htons(port);
addr.sin_addr.s_addr = INADDR_ANY;
ret = bind(lfd, (struct sockaddr*)&addr, sizeof addr);
```

4. 设置监听: `ret = listen(lfd, 128);`

5. 封装 listener: `listener->lfd = lfd; listener->port = *port*;`

服务器启动

```
threadPoolRun(server->threadPool);
channel = channelInit(server->listener->lfd, ReadEvent,
acceptConnection, NULL, NULL, server);
eventLoopAddTask(server->mainLoop, channel, ADD);
eventLoopRun(server->mainLoop);
```

启动线程池

创建 `channel` 实例，其监听的读事件的回调函数是接受客户端的连接请求

```
accept(server->listener->lfd, NULL, NULL);
```

然后注册到主反应堆中，并启动主反应堆